一、AUTH模块

二、教学目标

1、理解Auth模块的核心功能

2、掌握Auth模块的实践应用

3、了解项目开发全流程

4、理解项目开发中的表结构设计

三、auth模块

Django 提供了一个内置的 `django.contrib.auth` 模块（通常称为 "auth" 模块），用于处理用户认证、权限和授权等

3.0.1 用户模型 (`User`)

- 存储用户名、密码、邮箱等信息。

迁移数据库

```
python manage.py migrate
```

进行数据库迁移是会自动生成 10多张表,其中有一张名为 auth_user 的表.

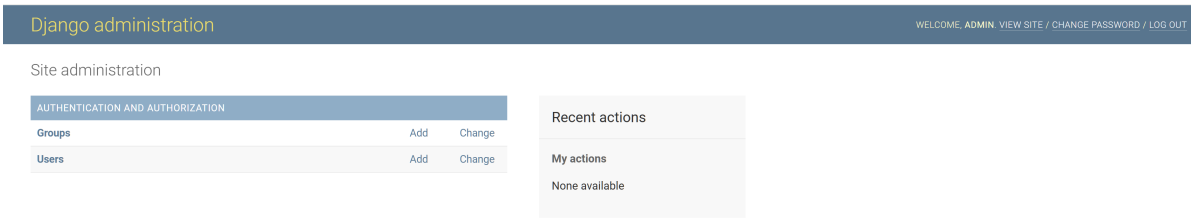
创建超级用户

```
python manage.py createsuperuser
```

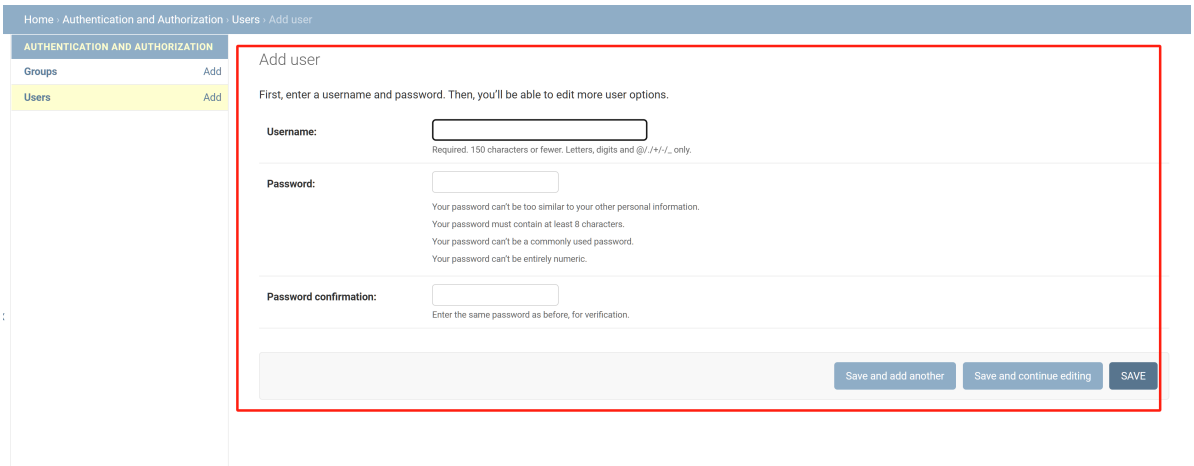
```
Username (leave blank to use 'dell'): admin    用户名
Email address:  邮箱
Password:       密码
Password (again):  确认密码
                                                提示密码太过于简单
This password is too short. It must contain at least 8 characters.
This password is too common.
This password is entirely numeric.
                                                是否创建
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.
```

创建之后就可以通过admin 路径进入到 django的后台管理

```
http://127.0.0.1:8000/admin/
```



进入到User



提示：不建议在这里创建用户 因为在admin后台创建用户的时候要通过4道验证

3.0.1 认证（登录/登出）

- `authenticate()`：验证用户凭据。
- `login(request, user)`：登录用户（创建会话）。
- `logout(request)`：登出用户（销毁会话）。

urls.py

```
#登录
path('login/',views.login),
#主页
path('home/',views.home)
#退出
path('logout/',views.logout,name='logout')
```

app01 - > views.py

```
from django.shortcuts import render,redirect

# Create your views here.
from django.contrib import auth

def login(request):
    if request.method == 'POST':
        username = request.POST.get('username')
        password = request.POST.get('password')

        # 1 怎么去auth_user查询用户是否存在
        # 2 auth_user表中的密码时是加密的 怎么是比对?
        user_obj=auth.authenticate(username=username,password=password)
        print(user_obj) #返回查找的用户对象
        #print(user_obj.username)
        #print(user_obj.password)
        #如果用户名和密码正确会返回查找的用户对象,
        #如果密码不正确,或者数据不符合的情况下会返回 None
        if user_obj:
            #保存登录的状态 类似于request.sesssion[key] = user_obj
            #只要执行了auth.login()方法,之后在任何地方
            #都可以通过request.user获取到用户对象
            auth.login(request,user_obj)
            #跳转首页
            return redirect('/home/')

    return render(request,'login.html')


def home(request):
    user = request.user
    #判断是否登录 如果登录了才进入主页
    if request.user.is_authenticated:
        return render(request, 'home.html',locals())
    else:
        return redirect('/login/')

def logout(request):

    auth.logout(request)  # 用户登出
    return redirect('/login/')
```

login.html

```
<body>
<form action="" method="post">
    {% csrf_token %}
    <p>用户名:  <input type="text" name="username"></p>
    <p>密码:  <input type="text" name="password"></p>
    <input type="submit">


</form>

</body>
```

home.html

```
<body>
<p>首页</p>
<p>欢迎{{ user }}</p>
<p><a href="{% url 'logout' %}">退出登录</a></p>
</body>
```

注意:如果用户登录之后,返回的就是登录的用户对象如果用户未登录,返回的是 AnonymousUser,所以不能通过 request.user 判断用户是否登录

3.0.1 授权

- **装饰器**：限制视图访问。

```
#导入模块
from django.contrib.auth.decorators import login_required
```

views.py

```
@login_required(login_url='/login/') #登录认证装饰器 没有登录就跳转到登录页面
def home(request):
    user = request.user
    return render(request, 'home.html',locals())


def login(request):
    if request.method == 'POST':
        ......
        if user_obj:
            ......
            url = request.GET.get('next') or '/home/'
            #跳转首页
            return redirect(url)
```

思考一下：假如有很多个视图函数都要用到登录认证装饰器的login_url怎么办?

@login_required(login_url='/login/')

解决方案:

全局配置

setting.py

```
#登录认证装饰器URL全局配置
LOGIN_URL= '/login/'
```

装饰器的括号内传入就是需要的登录页面的url也可以在 settings.py 文件中通过全局配置,之后需要登录跳转都会到这个页面

如果当前视图函数上再写login_url该执行那个?@login_required(login_url='/xxx/') **就近原则**

3.0.1 修改密码

修改的密码是auth_user表的密码 想一下怎么修改?

urls.py

```
 path('set_password/',views.set_password)
```

views.py

```
@login_required
def set_password(request):

    if request.method =='POST':
        old_password = request.POST.get('old_password')
        new_password = request.POST.get('new_password')
        confirm_password = request.POST.get('confirm_password')
        #先验证两次登录密码是否一致
        if new_password == confirm_password:
        #验证旧密码是否正确 check_password自动给旧密码加密并且校验
            is_right = request.user.check_password(old_password)
            if is_right:
                #修改密码 set_password 相当于修改对象的属性
                request.user.set_password(new_password)
                #操作数据库
                request.user.save()

                return redirect('/login/')

    return render(request,'set_password.html',locals())
```

set_password.html

```
<form action="" method="post">
    {% csrf_token %}
    <p>用户名: {{ request.user.username }}</p>
    <p>旧密码: <input type="text" name="old_password"> </p>
    <p>新密码: <input type="text" name="new_password"> </p>
    <p>确认密码: <input type="text" name="confirm_password"> </p>

    <input type="submit">
</form>
```

3.0.1 注册功能

实际上就是往auth_user表中添加用户

urls.py

```
path('register/',views.register)
```

app01 -> views.py

```
def register(request):

    if request.method == 'POST':
        username = request.POST.get('username')
        password = request.POST.get('password')
        confirm_password = request.POST.get('confirm_password')

        if password == confirm_password:
            #使用auth包中的User
            #create() 创建的用户对象,密码不能加密
            #User.objects.create(username=username,password=password)
            #create_superuser() 密码是加密的 创建是超级用户

 #User.objects.create_superuser(username=username,password=password,email='123@q
q.com')#邮箱是必填的
            #create_user()创建普通用户 密码是加密的 不是超级用户
            User.objects.create_user(username=username,password=password)
            #注册成功跳转到
            return redirect('/login/')

    return render(request,'register.html')
```

- create():创建不能给密码加密 也不是超级用户
- create_superuser():创建超级用户 密码是 加密的 是超级用户
- create_user():创建普通用户 密码是 加密的 不是超级用户

四、项目开发

4.0.1 模仿博客园的个人博客网站

1.需求分析

2.项目设计

3.分组开发

4.测试

5.交付上线

BBS表设计

一个项目最重要的是前期的表设计,只要将表结构设计好了,后续功能的书写就会很流畅

- **用户表**
  - auth_user(继承AbstractUser)
  - 自己创建一个auth_user表继承后就可以和auth模块当中的功能相当于合并
  - 扩展
    - phone 手机
    - avatar 用户头像
- **个人站点表**
  - site_name 站点名称
  - site_title 站点标题
  - site_theme 站点样式
  - 外键字段: 一对一用户表

- **文章分类表**
  - name 分类的名称
  - 外键字段: 一对多个人站点表
- **文章标签表**
  - name 标签名称
  - 外键字段: 一对多个人站点表
- **文章表**
  - title 文章标题
  - desc 文章简介
  - content 文章内容
  - create_time 发布时间
  - up_num 记录点赞数
  - down_num 记录点踩数

    - comment_num 记录评论数

    - 外键字段: 一对多文章分类表
    - 外键字段: 一对多个人站点表
    - 外键字段: 多对多文章标签表

- **点赞点踩表**
    - 都是外键关联表 - 记录哪个用户给那篇文章点了赞,点了踩
    - user 用户
    - article 文章
    - is_up 是否是点赞还是点踩 传入0或1
- **评论表**
    - 记录哪个用户给那篇文章评论了什么内容
    - user 用户
    - article 文章
    - content 评论内容
    - parent 根评论是谁 (自关联,用来关联自己)
        - 对文章的评论就是跟评论,对评论的评论就是子评论.

表对应的关系

用户表和个人站点表 一对一

文章分类和个人站点 一对多/多对一 的关系

文章标签和个人站点 一对多/多对一 的关系

文章表和文章分类表 一对多/多对一 的关系

文章表和个人站点 一对多/多对一 的关系

文章表和标签表 多对多的关系

五、作业

5.1 完成本天的案例